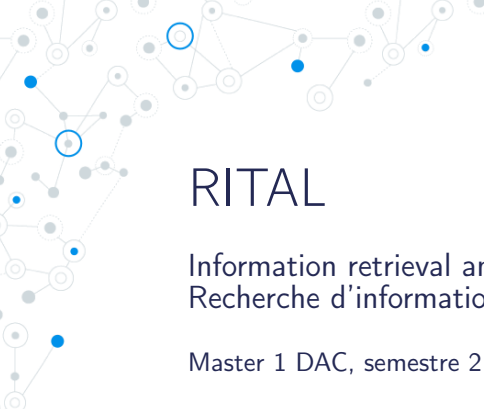




RITAL

Information retrieval and natural language processing
Recherche d'information et traitement automatique de la langue

Master 1 DAC, semestre 2

Nicolas Thome



Deep Learning in NLP

1 Deep Learning in NLP

- Convolutional Neural Networks (ConvNets)
- Recurrent Neural Networks (RNNs)
- Attention Models & Transformers

- Before 90's: linguistic, regex and simple statistical models (Naive Bayes)
- 90's: BoW, SVM and sequence models CRF
- 2000-2010: transition, seminal neural networks works for NLP
- 2010-now: renewal of deep learning & representation learning

1990 Matrix factorization :

- In NLP $\Rightarrow$ SVD / PCA

[Deerwester, 1990]

2005 First neural architecture for text representation

- an alternative to PLSA

[Keller, 2005]

2008 Convolutional Neural architecture for text

- precursor of modern architectures
- multi-tasks

[Collobert, 2008]

2012 The Word2Vec wave

- Qualitative & cheap word embeddings

[Mikolov, 2012]

2013 Manifest for representation learning

[Bengio, 2013]

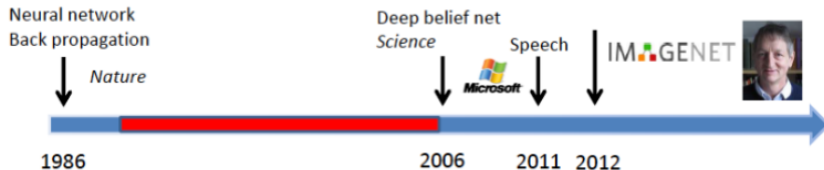
2015 Seq2seq paradigm

[Luong, 2015]

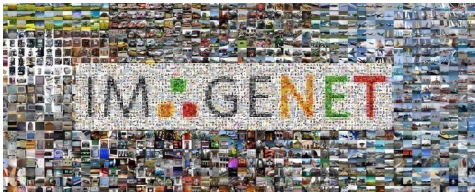
2017 Attention & Transformers

[Vaswani, 2017]

- 90's / 2000's: difficult to train large deep models on existing databases

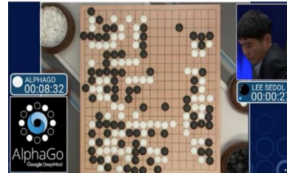
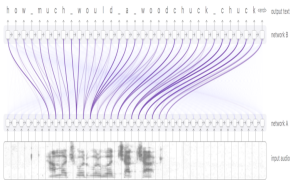
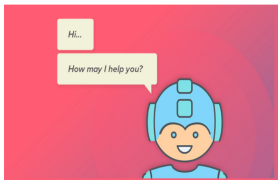


- ILSVRC'12: the deep revolution
⇒ outstanding success of ConvNets [KSH12]

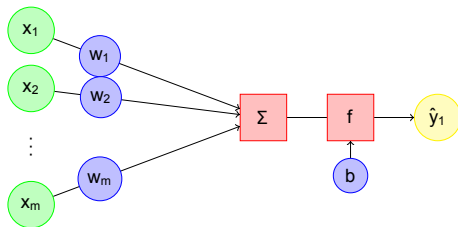


Rank	Name	Error rate	Description
1	U. Toronto	0.15315	Deep learning
2	U. Tokyo	0.26172	Hand-crafted
3	U. Oxford	0.26979	features and learning models.
4	Xerox/INRIA	0.27058	Bottleneck.

- Image classification, speech recognition
- chatbots, translation,
- Games, robotics



■ The formal Neuron

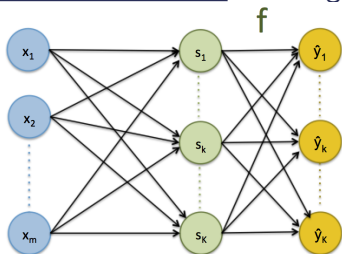


x_i : inputs
 w_i, b : weights
 f : activation function
 y : output of the neuron

$$y = f(w^T x + b)$$

Figure 1: Formal neuron – Credits: R. Herault

■ Neural Networks: Stacking several formal neurons $\Rightarrow$ Perceptron



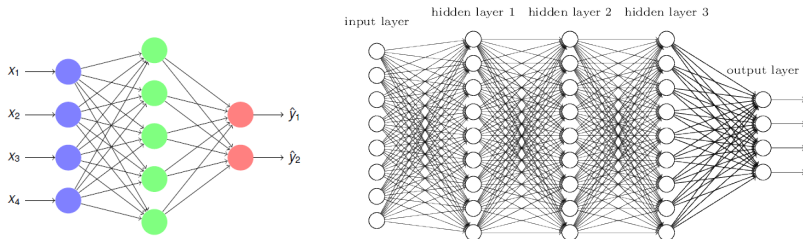
■ Soft-max Activation:

$$\hat{y}_k = f(s_k) = \frac{e^{s_k}}{\sum_{k'=1}^K e^{s_{k'}}}$$

$\Rightarrow$ Logistic Regression (LR) Model !

■ Multi-Layer Perceptron (MLP): Stacking layers of neural networks

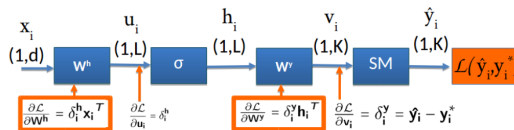
- More complex and rich functions / Logistic Regression (LR)
- **Neural network with one single hidden layer $\Rightarrow$ universal approximator [Cyb89]**



■ Basis of the "deep learning" field

- **Hidden layers: intermediate representations from data**
- **Can be learned with Backpropagation algorithm [Lec85, RHW86] (chain rule)**

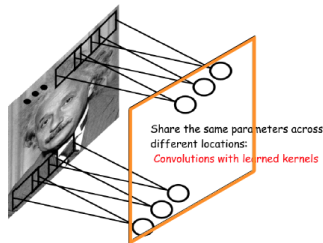
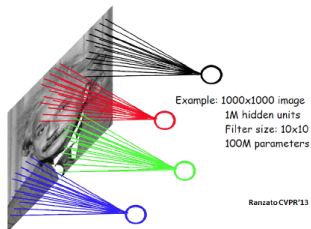
- $\mathcal{L}_{CE}(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{i=1}^N \ell_{CE}(\hat{y}_i, y_i^*) = -\frac{1}{N} \sum_{i=1}^N \log(\hat{y}_{c^*,i})$
- ℓ_{CE} smooth convex upper bound of $\ell_{0/1}$
 $\Rightarrow$ **gradient descent optimization**
- Gradient descent: $\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}}$
 $(\mathbf{b}^{(t+1)} = \mathbf{b}^{(t)} - \eta \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{b}})$
- Computing $\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = \frac{1}{N} \sum_{i=1}^N \frac{\partial \ell_{CE}}{\partial \mathbf{W}}$?
 $\Rightarrow$ **Backpropagation of gradient error!**
 $\Rightarrow$ Key Property: chain rule $\frac{\partial \mathbf{x}}{\partial \mathbf{z}} = \frac{\partial \mathbf{x}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{z}}$



1 Deep Learning in NLP

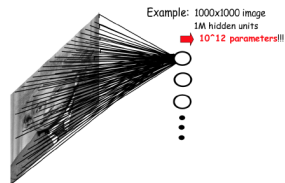
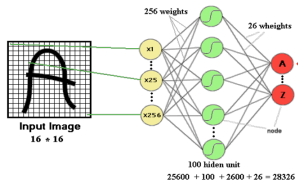
- Convolutional Neural Networks (ConvNets)
- Recurrent Neural Networks (RNNs)
- Attention Models & Transformers

- ConvNets: sparse connectivity + shared weights



- Local feature extraction ($\neq$ FCN)

- Overcome parameter explosion for FCN on images



■ Lookup table concept

■ W_0 matrix

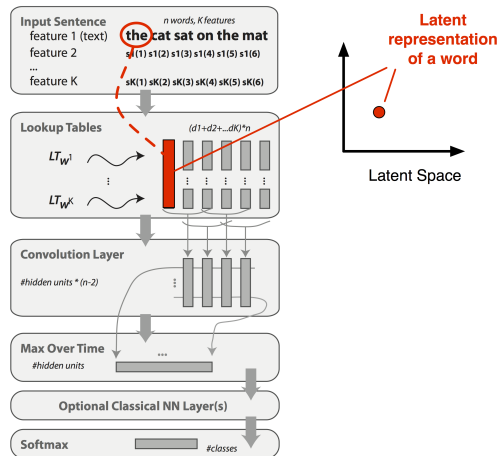
■ $\rightarrow$ Table of embeddings

■ 2008: state of the art on POS, NER, SRL

■ Quite difficult to set...

■ ... But open source + **open embeddings**

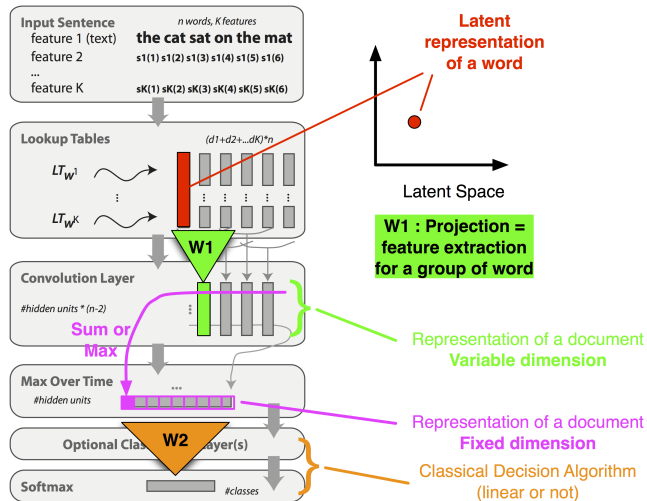
■ Based on torch... By Collobert



R. Collobert, J. Weston ICML 2008

A unified architecture for natural language processing: Deep neural networks with multitask learning

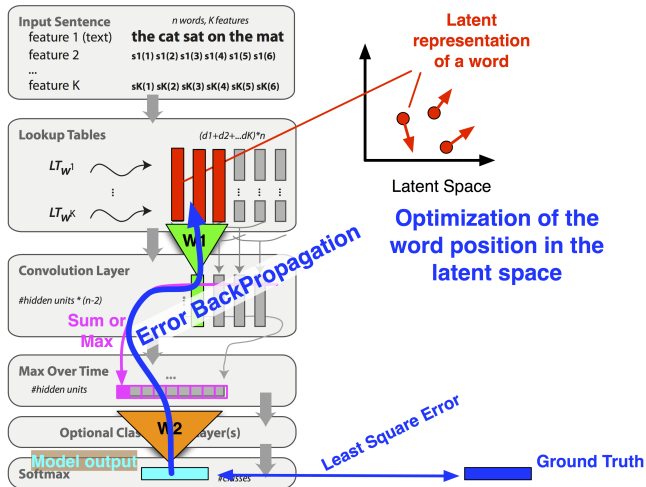
- Embedding matrix W_0
- Convolutional weights W_1
 - Limiting number of parameters
- Max pooling
 - Single document representation
- Prediction W_2
 - Trained for different tasks:
POS, NER, etc



R. Collobert, J. Weston ICML 2008

A unified architecture for natural language processing: Deep neural networks with multitask learning

- End-to-end training with backpropagation
- All weights W_2 , W_1 , W_0 optimized during training
- Based on error on a given target task



R. Collobert, J. Weston ICML 2008

A unified architecture for natural language processing: Deep neural networks with multitask learning

Several important informations

- Embedding are learned **keeping the sentence structure**
- Embeddings benefit from multi-tasks
- Learning is slow...

Our embeddings have been trained for about 2 months, over Wikipedia.

<https://ronan.collobert.com/senna/>

- ... But inference is fast & efficient

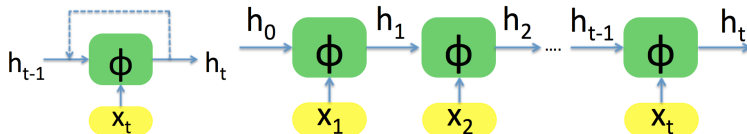
Task	Benchmark		Performance	Timing (s)
Part of Speech (POS)	(Toutanova et al, 2003)	(Accuracy)	97.29%	3
Chunking (CHK)	CoNLL 2000	(F1)	94.32%	2
Name Entity Recognition (NER)	CoNLL 2003	(F1)	89.59%	2
Semantic Role Labeling (SRL)	CoNLL 2005	(F1)	75.49%	36
Syntactic Parsing (PSG)	Penn Treebank	(F1)	87.92%	74

- + Costly embeddings have been made available to the community

1 Deep Learning in NLP

- Convolutional Neural Networks (ConvNets)
- Recurrent Neural Networks (RNNs)
- Attention Models & Transformers

- **RNN Cell:** $h_t = \phi(x_t, h_{t-1}) = f(Ux_t + Wh_{t-1} + b_h)$ [Elm90]
 - h_t : network memory up to time $t \Rightarrow$ Sequence processing
 - Same idea than HMMs, CRFs, but end-to-end training of representation

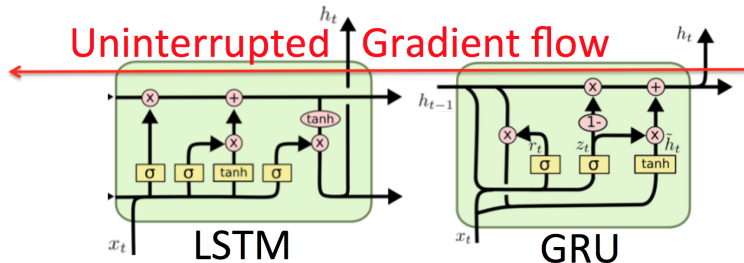


Folded RNN

Unfolded RNN

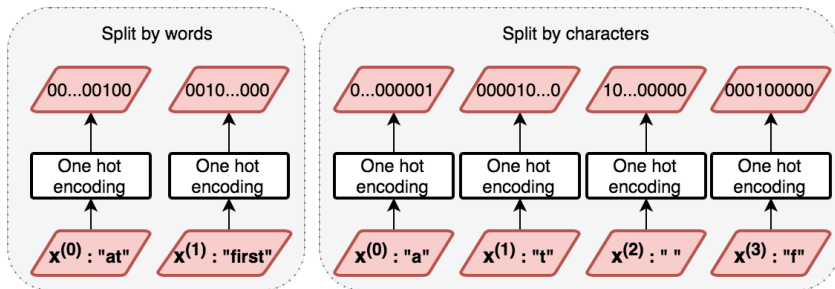
- Specific architectures for vanishing gradients

LSTM [HS97],
GRU [CvMG⁺14]



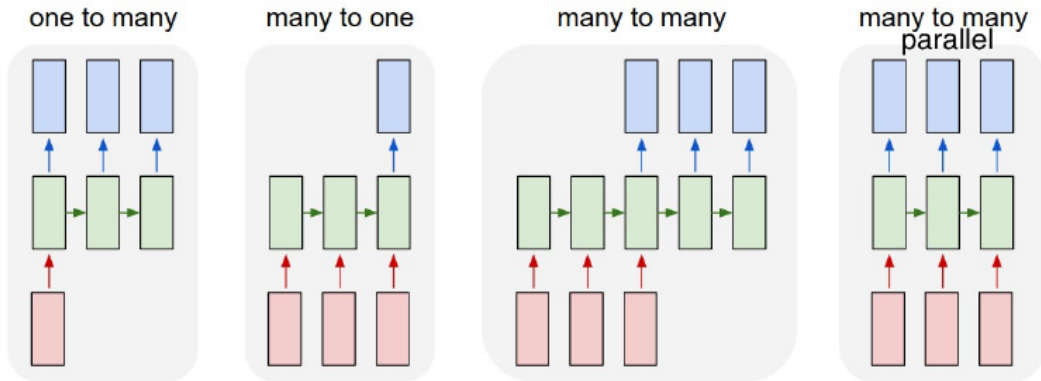
Deep NLP strategy:

- 1 Extracts text input, "tokens", e.g. characters or words
- 2 One hot encoding of tokens



- 3 Split the text into a "temporal" sequence
- 4 RNN to model the temporal structure
 - Option: use an embedding layer on top of one-hot encoding

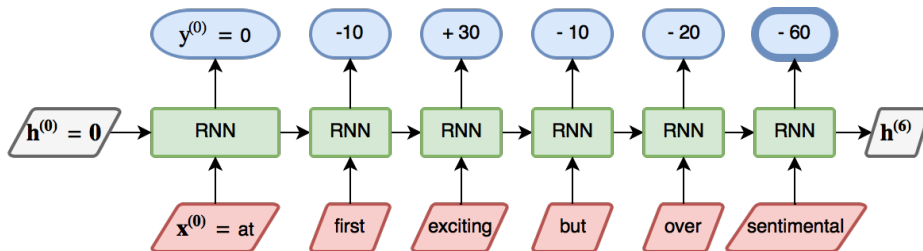
Different uses for different applications



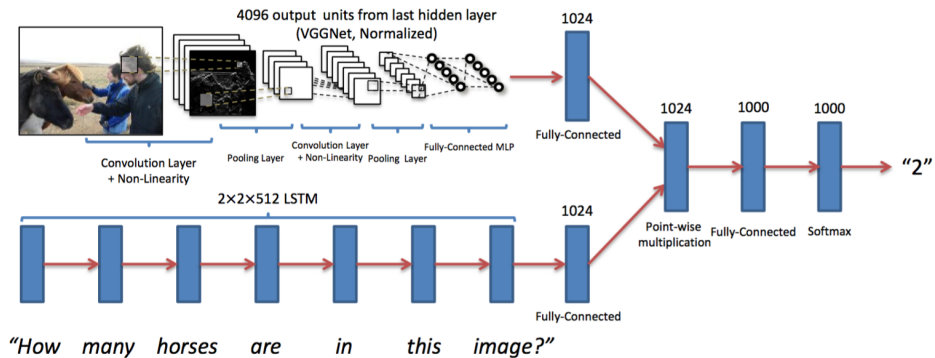
Karpathy's blog <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

■ Sentiment classification

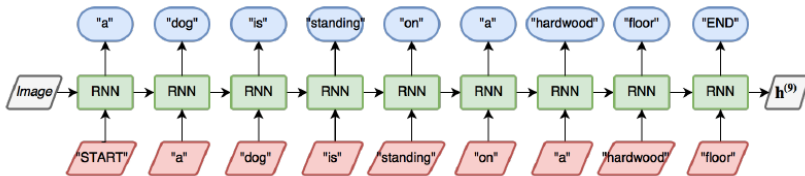
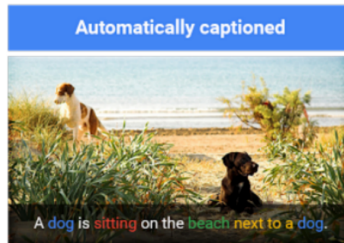
- Input: "At first exciting but over sentimental"
- Output: -60 = Bad review



■ Question Answering and Visual Question Answering (VQA)

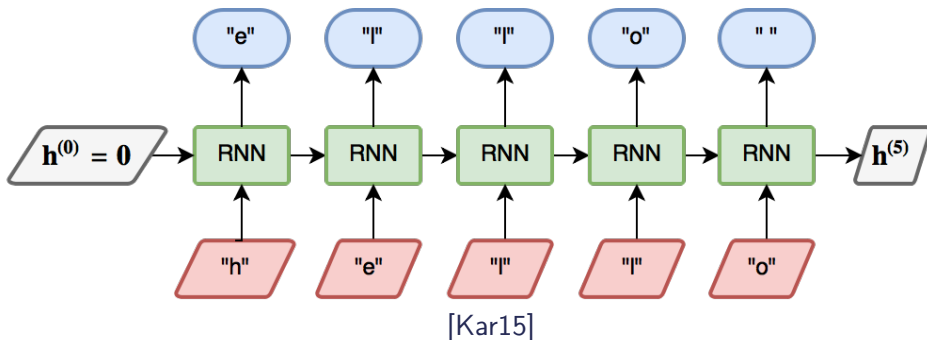


■ Image captioning



[KL15]

- **Text Generation**, e.g. Char-nn
 - Input sequence of characters (Token $\Leftrightarrow$ char)
 - Output: next character
- Many-to-many parallel
 - In practice: many-to-one: predict next character from previous (K) chars



- Char-nn: applied to raw text, e.g. poetry (practical session)
 - Char-nn: learns to correctly spell a given language, although semantic meaning of sentences more challenging
 - Capacity to learn language structural/syntactical rules
 - ⇒ applications for generating source code, e.g. wikipedia pages, XML, Latex, linux source code (C), etc
 - See [here](#) for other examples

Proof. Omitted. □

Lemma 0.1. *Let $\mathcal{C}$ be a set of the construction.*
Let $\mathcal{C}$ be a gerbe covering. Let $\mathcal{F}$ be a quasi-coherent sheaves of $\mathcal{O}$ -modules. We have to show that

$$\mathcal{O}_{\mathcal{C}_X} = \mathcal{O}_X(\mathcal{L})$$

Proof. This is an algebraic space with the composition of sheaves $\mathcal{F}$ on $X_{\text{étale}}$ we have

$$\mathcal{O}_X(\mathcal{F}) = \{ \text{morph}_1 \times_{\mathcal{O}_X} (\mathcal{G}, \mathcal{F}) \}$$

where $\mathcal{G}$ defines an isomorphism $\mathcal{F} \rightarrow \mathcal{F}$ of $\mathcal{O}$ -modules. □

Lemma 0.2. *This is an integer $\mathbb{Z}$ is injective.*

Proof. See Spaces, Lemma ?? □

Lemma 0.3. *Let S be a scheme. Let X be a scheme and X is an affine open covering. Let $\mathcal{U} \subset X$ be a canonical and locally of finite type. Let X be a scheme. Let X be a scheme which is equal to the formal complex.*

The following to the construction of the lemma follows.

Let X be a scheme. Let X be a scheme covering. Let

$$b : X \rightarrow Y' \rightarrow Y \rightarrow Y' \times_X Y \rightarrow X.$$

be a morphism of algebraic spaces over S and Y .

Proof. Let X be a nonzero scheme of X . Let X be an algebraic space. Let $\mathcal{F}$ be a quasi-coherent sheaf of $\mathcal{O}_X$ -modules. The following are equivalent

- (1) $\mathcal{F}$ is an algebraic space over S .
- (2) If X is an affine open covering.

Consider a common structure on X and X the functor $\mathcal{O}_X(\mathcal{U})$ which is locally of finite type. □

This since $\mathcal{F} \in \mathcal{F}$ and $x \in \mathcal{G}$ the diagram

is a limit. Then $\mathcal{G}$ is a finite type and assume S is a flat and $\mathcal{F}$ and $\mathcal{G}$ is a finite type f_x . This is of finite type diagrams, and

- the composition of $\mathcal{G}$ is a regular sequence,
- $\mathcal{O}_X$ is a sheaf of rings.

□

Proof. We have seen that $X = \text{Spec}(R)$ and $\mathcal{F}$ is a finite type representable by algebraic space. The property $\mathcal{F}$ is a finite morphism of algebraic stacks. Then the cohomology of X is an open neighbourhood of U . □

Proof. This is clear that $\mathcal{G}$ is a finite presentation, see Lemma ??.

A reduced above we conclude that U is an open covering of $\mathcal{C}$. The functor $\mathcal{F}$ is a "field"

$$\mathcal{O}_{X,\mathcal{F}} \rightarrow \mathcal{F}_Y \rightarrow \mathcal{I}(\mathcal{O}_{X_{\text{étale}}}) \rightarrow \mathcal{O}_{X,\mathcal{F}}^*(\mathcal{O}_{X,\mathcal{F}}^*)$$

is an isomorphism of covering of $\mathcal{O}_{X,\mathcal{F}}$. If $\mathcal{F}$ is the unique element of $\mathcal{F}$ such that X is an isomorphism.

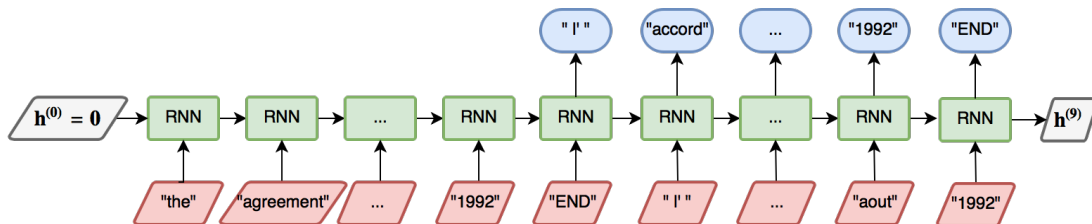
The property $\mathcal{F}$ is a disjoint union of Proposition ?? and we can filtered set of presentations of a scheme $\mathcal{O}_X$ -algebra with $\mathcal{F}$ are opens of finite type over S . □

If $\mathcal{F}$ is a scheme theoretic image points.

If $\mathcal{F}$ is a finite direct sum $\mathcal{O}_{X,\mathcal{F}}$ is a closed immersion, see Lemma ?? This is a sequence of $\mathcal{F}$ is a similar morphism.

■ Machine Translation text2text

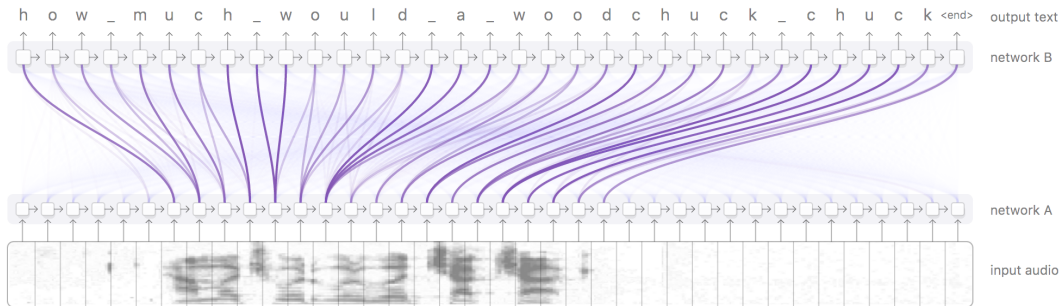
- Input: "The agreement on the European Economic Area was signed in August 1992."
- Output: "L'accord sur la zone économique européenne a été signé en août 1992."



[BCB14] [OC16]

■ Machine Translation speech2text

- Input : Audio mp3 (speech utterance)
- Output: "How much would a woodchuck chuck"

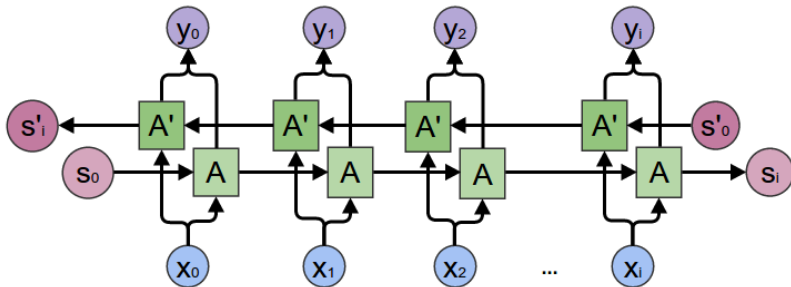


[CJLV15] [OC16]

LSTM

+ Sequential modeling

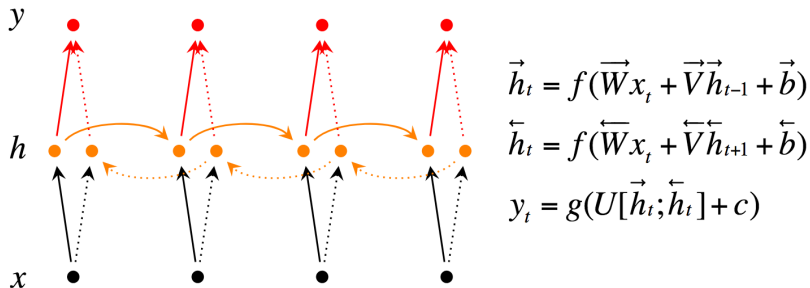
- Sequential dependencies ! = partial modeling



Bi-dimensional representation $[S_1, S'_1]$ is more powerful representation of the sentence S than each single representation.

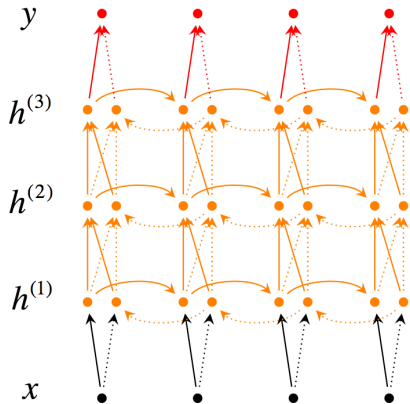
Classical notation: $s = [\vec{s}, \overleftarrow{s}]$

- For classification, incorporate information from words both preceding and following



$h = [\vec{h}; \overleftarrow{h}]$ now represents (summarizes) the past and future around a single token.

■ Deep Bi-directional RNNs



$$\vec{h}_t^{(i)} = f(\vec{W}^{(i)} h_t^{(i-1)} + \vec{V}^{(i)} \vec{h}_{t-1}^{(i)} + \vec{b}^{(i)})$$

$$\overleftarrow{h}_t^{(i)} = f(\overleftarrow{W}^{(i)} h_t^{(i-1)} + \overleftarrow{V}^{(i)} \overleftarrow{h}_{t+1}^{(i)} + \overleftarrow{b}^{(i)})$$

$$y_t = g(U[\vec{h}_t^{(L)}; \overleftarrow{h}_t^{(L)}] + c)$$

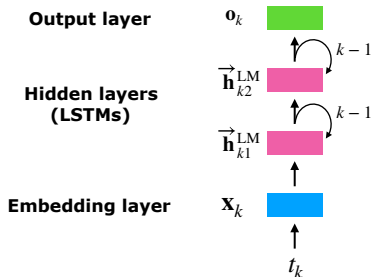
Each memory layer passes an intermediate sequential representation to the next.

Static word embeddings

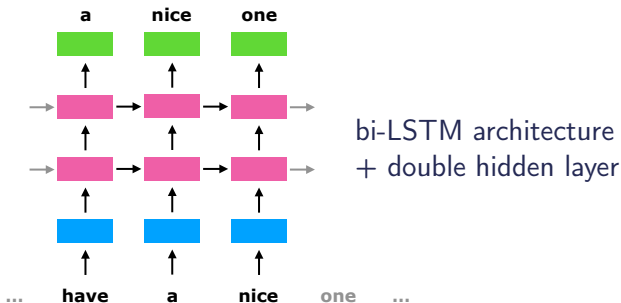


Mapping function = dynamic embeddings

The forward LM architecture



Expanded in the forward direction of k



M. E. Peters et al. arXiv 2018 Deep contextualized word representations

Static word embeddings



Mapping function = dynamic embeddings



ELMo represents a word t_k as a linear combination of corresponding hidden layers (inc. its embedding)

ELMo is a task specific representation. A down-stream task learns weighting parameters

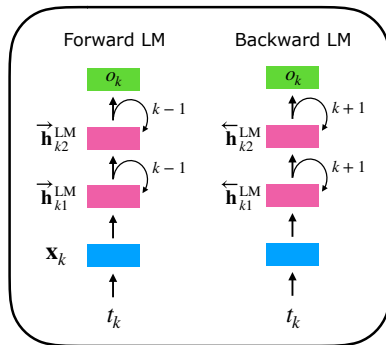
$$\text{ELMo}_k^{\text{task}} = \gamma^{\text{task}} \times \sum \left\{ \begin{array}{l} s_2^{\text{task}} \times \mathbf{h}_{k2}^{\text{LM}} \\ s_1^{\text{task}} \times \mathbf{h}_{k1}^{\text{LM}} \\ s_0^{\text{task}} \times \mathbf{h}_{k0}^{\text{LM}} \end{array} \right. \quad \left([\mathbf{x}_k; \mathbf{x}_k] \right)$$

Concatenate hidden layers

$[\vec{\mathbf{h}}_{kj}^{\text{LM}}; \overleftarrow{\mathbf{h}}_{kj}^{\text{LM}}]$

Unlike usual word embeddings, ELMo is assigned to every *token* instead of a *type*

biLMs

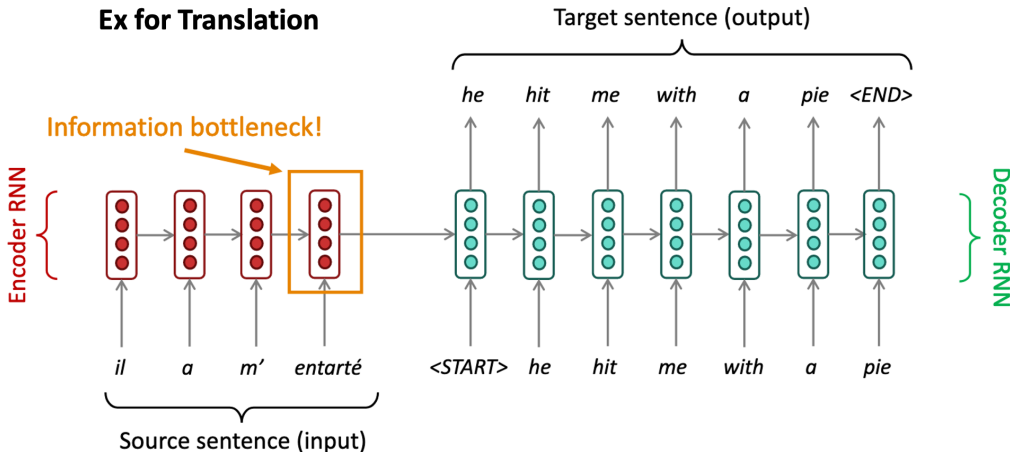


1 Deep Learning in NLP

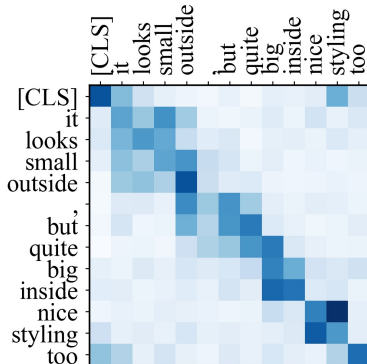
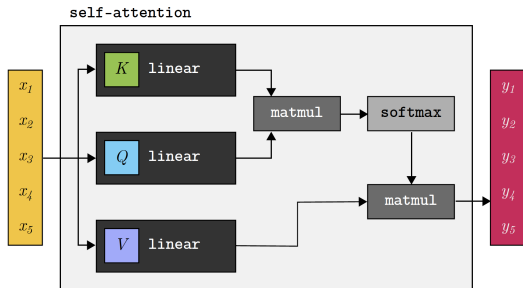
- Convolutional Neural Networks (ConvNets)
- Recurrent Neural Networks (RNNs)
- Attention Models & Transformers

- Global context: very important in NLP, **BUT ConvNets**: only local information
- **RNNs**: can in theory encode long-range dependencies **BUT**:
 - Vanishing gradients, bottleneck (last word representation), computation issues (parallelization)

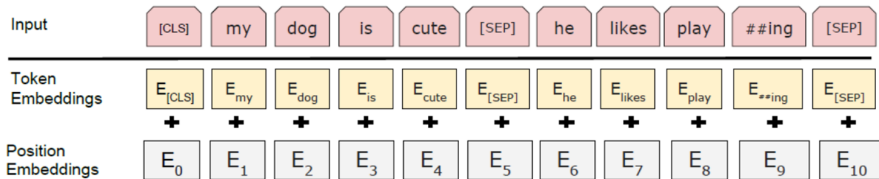
Ex for Translation



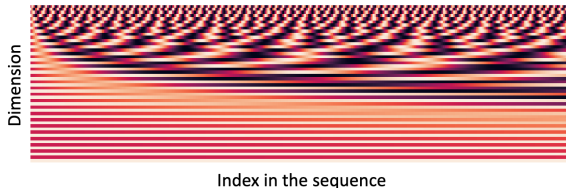
- Transformers: only fully connected layers ($\neq$ ConvNets, RNNs)
- Attention: for a set of input tokens $\mathbf{X} := \{\mathbf{x}_i\}_{i \in \{1; T\}}$, $\mathbf{x}_i \in \mathbb{R}^d$, learn 3 matrix weights:
 - $\mathbf{K} = \mathbf{X}\mathbf{W}_k$, $\mathbf{Q} = \mathbf{X}\mathbf{W}_q$, $\mathbf{V} = \mathbf{X}\mathbf{W}_v$ ($\mathbf{K}$ Keys, $\mathbf{Q}$ Queries, $\mathbf{V}$ Values)
 - $\mathbf{A} = \text{Softmax}_T\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)$ and $\mathbf{Y} = \mathbf{A}\mathbf{V}$
 - $y_j = \sum_{i=1}^T v_i a_{i,j}$
- [CLS] token: global description of doc



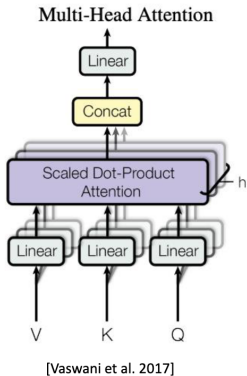
- Fully connected layers: permutation invariant $\Rightarrow$ no info about each token position!
- Adding positional encoding, handcrafted (sinusoid), or learned



$$p_i = \begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*d/2/d}) \\ \cos(i/10000^{2*d/2/d}) \end{pmatrix}$$

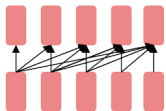


- **High-Level Idea:** Let's perform self-attention multiple times in parallel and combine the results.



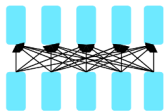
Wizards of the Coast, Artist: Todd Lockwood

Credit: Anna Goldie



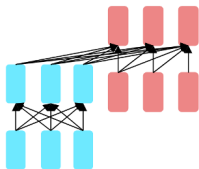
Decoders

- Language models!
- Nice to generate from; can't condition on future words
- **Examples:** GPT-2, GPT-3, LaMDA



Encoders

- Gets bidirectional context – can condition on future!
- Wait, how do we pretrain them?
- **Examples:** BERT and its many variants, e.g. RoBERTa

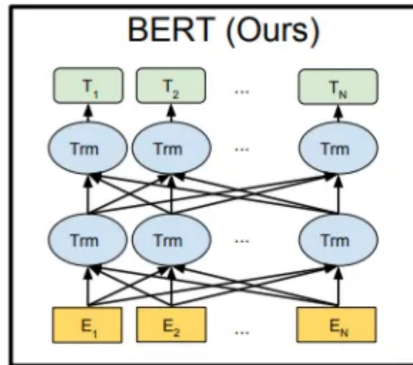
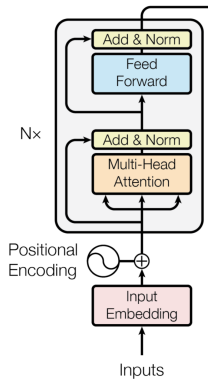


**Encoder-
Decoders**

- Good parts of decoders and encoders?
- What's the best way to pretrain them?
- **Examples:** Transformer, T5, Meena

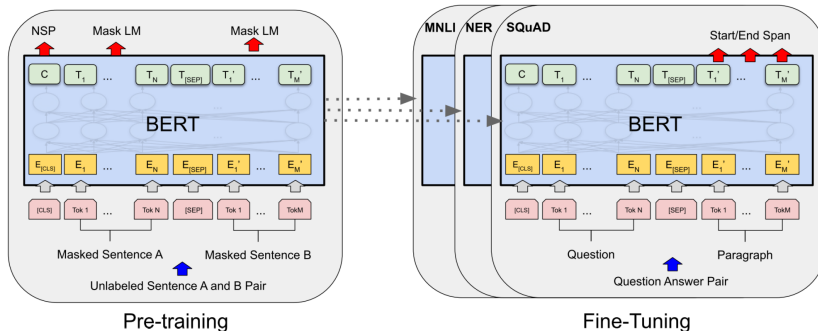
Credit: Anna Goldie

- Transformer block: MHA (+norm) + FFN (+norm), & residual connections
- BERT [DCLT18]: Bidirectional Encoder Representations from Transformer
 - BERT: deep cascade of transformers T_m
 - Context-based embedding ($\neq$ Word2vec, Glove), bi-directional embedding ($\neq$ ELMO)

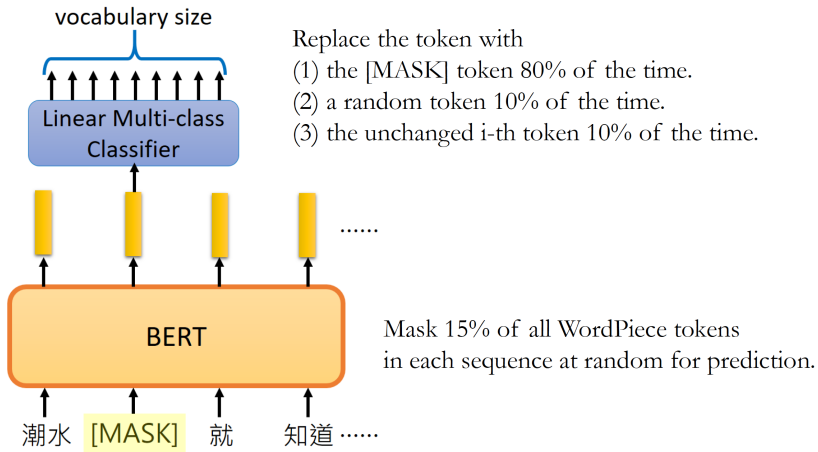


BERT: pre-training & fine-tuning

- Pre-training on unlabeled data: huge data ($\sim$ ImageNet in vision)
 - Next Sequence Prediction (NSP)
 - Masked Language Model (MLM)
- Fine-tuning on downstream task
 - [CLS] token: can be used to provide a global sentence representation
 - Leverages learned attention for effective pooling (better than avg, max)

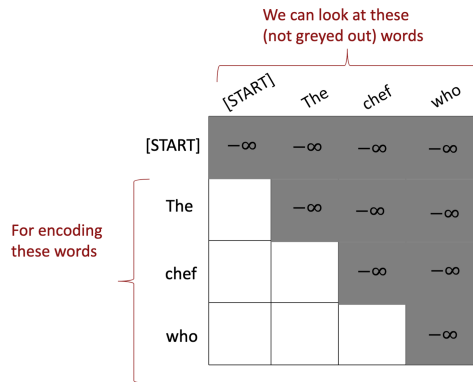
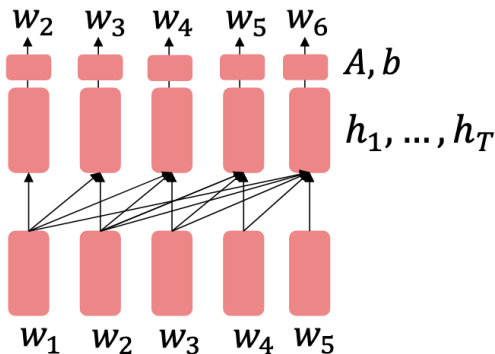


Credit: Y. Fang

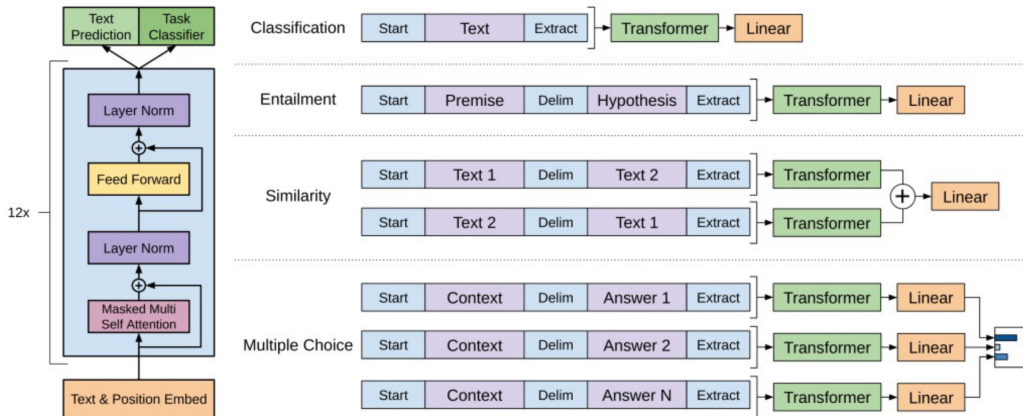


Credit: Y. Fang

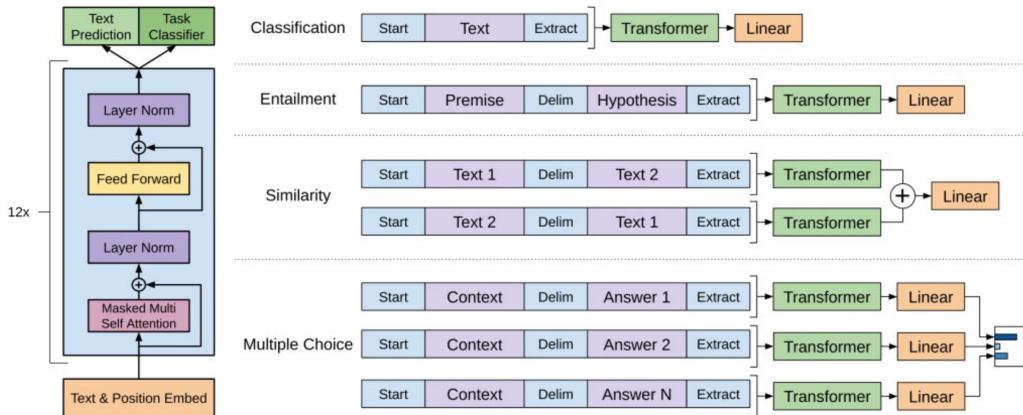
- Decoder: generative language models to predict next word: $P_{\theta}(\mathbf{w}_t | (\mathbf{w}_{t-1}, \dots, \mathbf{w}_1))$
- Issue: How do we keep the decoder from "cheating" (look ahead and "see" the answer)?
 - Solution: Masked Attention: hide (mask) information about future tokens from the model



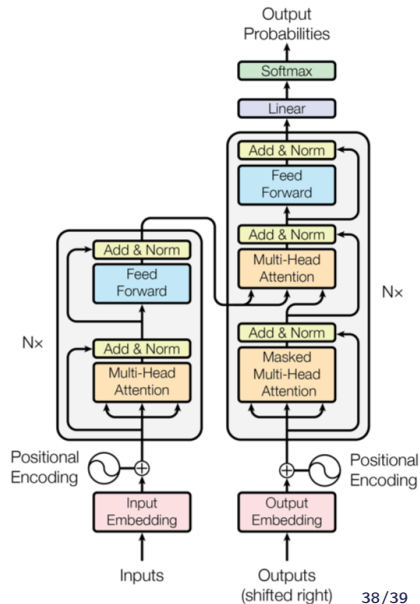
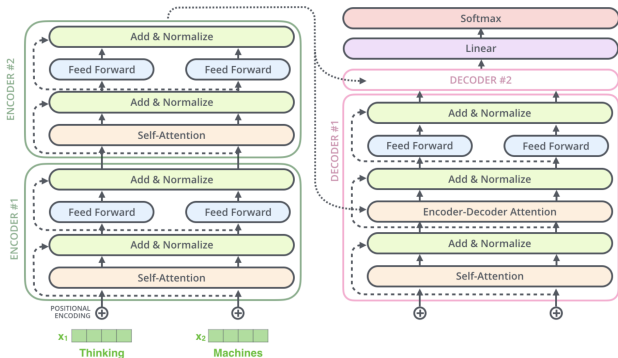
- Generative Pre-trained Transformer (GPT) [RNSS18]
- 12 layers: 768-dimensional hidden states, 3072-dimensional feed-forward hidden layers
- Byte-pair encoding with 40,000 merges, trained on BooksCorpus (over 7000 unique books)
- Contains long spans of contiguous text, for learning long-distance dependencies.



- GPT: Big success in pre-training a decoder
- GPT-2: larger version trained on more data $\Rightarrow$ convincing samples of natural language
- GPT-3: 175 billion parameters (10x more than GPT-2)

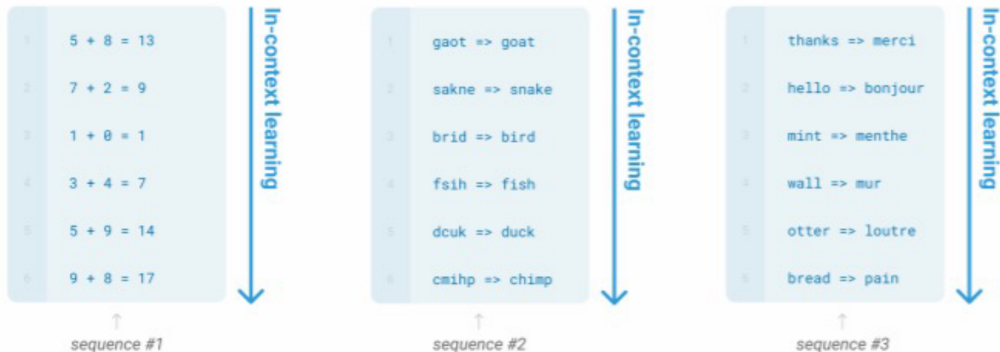


- Encoder+Decoder: Seq2Seq with transformers
- Famous example for translation: "attention is all you need" [VSP⁺17]
- Cross-attention between encoder and decoder



- Transformers: revolution in NLP
- Mediatic impact, see GPT-3
- Prompt ('in-context') learning: emerging (and not fully understood) behavior

Learning via SGD during unsupervised pre-training



- 
- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio.
Neural machine translation by jointly learning to align and translate.
CoRR, abs/1409.0473, 2014.
- 
- William Chan, Navdeep Jaitly, Quoc V. Le, and Oriol Vinyals.
Listen, attend and spell.
CoRR, abs/1508.01211, 2015.
- 
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio.
Learning phrase representations using rnn encoder-decoder for statistical machine translation, 2014.
cite arxiv:1406.1078Comment: EMNLP 2014.
- 
- George Cybenko.
Approximation by superpositions of a sigmoidal function.
Mathematics of control, signals and systems, 2(4):303–314, 1989.
- 
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova.
BERT: pre-training of deep bidirectional transformers for language understanding.
CoRR, abs/1810.04805, 2018.
- 
- Jeffrey L. Elman.
Finding structure in time.
COGNITIVE SCIENCE, 14(2):179–211, 1990.



Sepp Hochreiter and Jürgen Schmidhuber.
Long short-term memory.
Neural Comput., 9(8):1735–1780, November 1997.



Andrej Karpathy.
The unreasonable effectiveness of recurrent neural networks, 2015.



Andrej Karpathy and Fei-Fei Li.
Deep visual-semantic alignments for generating image descriptions.
In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*, pages 3128–3137, 2015.



Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton.
Imagenet classification with deep convolutional neural networks.
In *Advances in neural information processing systems*, pages 1097–1105, 2012.



Yann Lecun.
Une procédure d'apprentissage pour reseau a seuil asymmetrique (A learning scheme for asymmetric threshold networks), pages 599–604.
1985.



Chris Olah and Shan Carter.
Attention and augmented recurrent neural networks.
Distill, 2016.



D.E. Rumelhart, G.E. Hinton, and R.J. Williams.
Learning representations by back-propagating errors.
Nature, 323:533–536, October 1986.



Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever.
Improving language understanding by generative pre-training.
2018.



Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin.
Attention is all you need.

In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008. Curran Associates, Inc., 2017.